Taller 2: Redes Neuronales Multicapa (MLP) y Backpropagation

Informe de Resultados

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiante: Sergio Santiago Duarte Rojas
Código: 20261595002

Fecha: 11 de octubre de 2026

Universidad Distrital Francisco José de Caldas

Índice

1. Introducción **2**

2. Marco teórico (resumen) **2**
2.1. Arquitectura del perceptrón multicapa 2
2.2. El algoritmo de retropropagación 2
2.3. Por qué la XOR necesita una capa oculta 3
2.4. Sobreajuste y parada temprana 3

3. Metodología **3**
3.1. Implementación 3
3.2. Arquitecturas y parámetros evaluados 4
3.3. Conjuntos de datos y particiones 4

4. Resultados **4**
4.1. Problema XOR 4
4.2. Tasa de aprendizaje, momento y pesos iniciales sobre la XOR 5
4.3. Activación en la capa oculta frente a la capa de salida 6
4.4. Clasificación multiclase en Iris 7
4.5. Wine, Breast Cancer Wisconsin y billetes en distintas particiones 8
4.6. Análisis de sobreajuste (overfitting) 9

5. Análisis y discusión **12**
5.1. Por qué el MLP resuelve la XOR y los modelos del Taller 1 no 12
5.2. Efecto del número de neuronas ocultas y de la tasa de aprendizaje 12
5.3. Proporción de partición y generalización 12
5.4. Evidencia de sobreajuste y su mitigación 12

6. Conclusiones **13**

7. Referencias **13**

A. Tablas complementarias **14**

B. Código de los bloques completados **14**

1. Introducción

El Perceptrón y el Adaline del Taller 1 no resolvieron la XOR, porque una neurona traza un solo hiperplano y la XOR necesita dos. Este taller implementa un perceptrón multicapa entrenado por retropropagación y comprueba qué gana la red al agregar la capa oculta.

Se completaron las cuatro funciones pendientes del *live script* del docente sin cambiar nombres ni argumentos, y sobre ellas se montaron los experimentos de los numerales 3 a 6. La XOR de dos y tres entradas se compara con el Taller 1 y sobre ella se estudian el barrido de tasa y momento de la guía de laboratorio (2007), la escala de los pesos iniciales y la activación de cada capa. Iris se prueba con cuatro arquitecturas y tres tasas. Wine, Breast Cancer Wisconsin y billetes se evalúan en cuatro particiones frente al Perceptrón y el Adaline, y en Breast Cancer se analiza el sobreajuste y la parada temprana que lo corrige.

2. Marco teórico (resumen)

2.1 Arquitectura del perceptrón multicapa

Un perceptrón multicapa organiza las neuronas en capas sucesivas. La capa de entrada solo distribuye el patrón, cada capa oculta calcula una suma ponderada de las salidas de la capa anterior y le aplica una función de activación, y la capa de salida entrega la respuesta de la red. Para la neurona j de la capa k el cálculo es el de la ecuación (1), donde $a^{(0)}$ es la entrada x.

$$net_j^{(k)} = \sum_i w_{ji}^{(k)} a_i^{(k-1)} + b_j^{(k)}, \qquad a_j^{(k)} = f\left(net_j^{(k)}\right) \tag{1}$$

Este recorrido desde la entrada hasta la salida se llama propagación hacia adelante. La diferencia con la neurona única del Taller 1 está en que la función f de las capas ocultas debe ser derivable y no lineal, con la salvedad de la ReLU, que no es derivable en cero y a la que ahí se le asigna derivada cero. El taller ofrece tres opciones, la sigmoide $1/(1+e^{-net})$ que produce valores entre cero y uno, la tangente hiperbólica que produce valores entre menos uno y uno, y la ReLU, que vale *net* cuando es positivo y cero en caso contrario.

2.2 El algoritmo de retropropagación

La red se entrena minimizando el error cuadrático $E = \frac{1}{2}\sum(d(x)-Y(x))^2$ por descenso de gradiente, igual que el Adaline. La novedad es que ahora hay pesos en capas que no ven directamente el error, y para corregirlos hace falta repartir ese error hacia atrás. El reparto se hace con un término δ por neurona. En la capa de salida el término se calcula con el error observado, ecuación (2), y en cada capa oculta se obtiene a partir de los δ de la capa siguiente pesados por las conexiones que las unen, ecuación (3), donde $\odot$ es el producto elemento a elemento y W^T la traspuesta de la matriz de pesos.

$$\delta^{(L)} = (d(x) - Y(x)) \odot f'\left(net^{(L)}\right) \tag{2}$$

$$\delta^{(k)} = \left(W^{(k+1)T}\delta^{(k+1)}\right) \odot f'\left(net^{(k)}\right) \tag{3}$$

Con los δ calculados, cada peso se corrige en proporción al δ de la neurona que alimenta y a la activación que recibe, ecuación (4), donde η es la tasa de aprendizaje. El coeficiente de momento β, que la guía de laboratorio del curso (2007) propone variar entre 0 y 1, agrega una fracción del

incremento anterior, lo cual acelera el avance cuando los incrementos sucesivos apuntan en la misma dirección.

$$\Delta W^{(k)} = \eta\, \delta^{(k)} \left(a^{(k-1)}\right)^T + \beta\, \Delta W^{(k)}_{\text{anterior}}, \qquad W^{(k)} \leftarrow W^{(k)} + \Delta W^{(k)} \tag{4}$$

Con una sola capa, activación lineal y sin momento, las ecuaciones (2) y (4) se reducen a la Regla Delta del Taller 1. La retropropagación es entonces la extensión natural de ese algoritmo a redes con capas ocultas.

2.3 Por qué la XOR necesita una capa oculta

Una neurona con función escalón divide el espacio de entrada en dos regiones separadas por un hiperplano. La XOR asigna la clase uno a $(0, 1)$ y $(1, 0)$ y la clase cero a $(0, 0)$ y $(1, 1)$, y ninguna recta deja a cada lado los puntos de una clase. Con una capa oculta la red traza dos rectas y las combina. Dos neuronas ocultas bastan en principio, una que se activa cuando alguna entrada vale uno y otra cuando ambas valen uno, y la salida responde uno solo cuando la primera está activa y la segunda no. Rumelhart, Hinton y Williams (1986) mostraron que una capa oculta entrenada por retropropagación resuelve exactamente este tipo de problema.

2.4 Sobreajuste y parada temprana

Una red con suficientes pesos y épocas puede aprender de memoria los datos de entrenamiento. El síntoma es que el error de entrenamiento sigue bajando mientras el error sobre datos no vistos deja de bajar y sube. Eso es el sobreajuste (Haykin, 2009). La parada temprana vigila el error sobre un conjunto de validación y detiene el entrenamiento cuando lleva cierto número de épocas sin mejorar, conservando los pesos del mejor punto. Es el criterio de parada por épocas sin mejora que pide el numeral 2.

3. Metodología

3.1 Implementación

`init_mlp` crea pesos aleatorios pequeños y sesgos en cero. `forward_mlp` aplica la ecuación (1) capa por capa y guarda las sumas ponderadas y las activaciones. `backward_mlp` calcula los δ de las ecuaciones (2) y (3) y actualiza pesos y sesgos con la ecuación (4). `activation_deriv` entrega la derivada de cada activación. La activación oculta se indica con un nombre para todas las capas o con una lista con una por capa. La plantilla usaba `net` sin haberla creado, así que se agregó la llamada a `init_mlp`. El código va en el Anexo B.

`train_mlp` sigue el ciclo de la plantilla, con la diferencia de que mide el error al final de cada época sobre todo el conjunto. Calcula además el error de validación e implementa la parada por épocas sin mejora con el campo `patience`. Las demás funciones predicen la clase, cargan cada conjunto, parten con semilla y escalan al rango cero a uno. Las de partición y escalado vienen del Taller 1. `Dataset_train_test_T2` es la versión para Wine y Breast Cancer del script de particiones del Taller 1, con la misma semilla que el experimento. El Perceptrón y el Adaline se incluyen sin cambios, y con más de dos clases se entrena una neurona por clase y decide la mayor suma ponderada.

3.2 Arquitecturas y parámetros evaluados

La Tabla 1 recoge las configuraciones. Salvo donde se indica, se usan momento β igual a cero, salida sigmoide, pesos iniciales de escala 0.5, tasa 0.3 y el error objetivo de la plantilla, 0.001. En la XOR cada configuración se repite con cinco semillas. Las arquitecturas entre corchetes tienen dos capas ocultas. El Perceptrón y el Adaline conservan los parámetros del Taller 1, tasa 0.1 y 100 épocas el primero, tasa 0.05 y 200 épocas el segundo. Las tablas y figuras llaman MSE al error E de la sección 2.2 promediado sobre los patrones.

Tabla 1: Configuraciones de MLP evaluadas. La parada es por error objetivo o por número de épocas sin mejora en validación

Experimento	Ocultas	Activación oculta	η	Momento	Épocas	Parada
XOR	2, 4, 8, [4 4]	sigmoide, tanh, ReLU	0.1 a 2	0 a 1	5000	error 0.001
Iris	3, 6, 12, [8 4]	sigmoide	0.05, 0.1, 0.3	0	1000	error 0.001
Conjuntos reales	8	sigmoide	0.3	0	500	error 0.001 o 30 sin mejora
Sobreajuste	2, 8, 32, 64	sigmoide	0.1	0	200 a 1500	ninguna o 50 sin mejora

3.3 Conjuntos de datos y particiones

La XOR de n entradas vale uno cuando un número impar de entradas vale uno, 4 patrones con dos entradas y 8 con tres. Iris tiene 150 flores, cuatro medidas y tres especies, codificadas con una salida por especie. Wine tiene 178 vinos de tres cultivares con trece medidas. Breast Cancer Wisconsin Diagnostic tiene 569 tumores con treinta medidas, 357 benignos y 212 malignos. Billetes es el conjunto del Taller 1, 1372 patrones, cuatro medidas y dos clases. Wine proviene de Aeberhard y Forina (1992), Breast Cancer de Wolberg et al. (1993) y billetes de Lohweg (2012), todos del repositorio UCI, e Iris es el conjunto de Fisher (1936). La Tabla 2 resume los tamaños.

Tabla 2: Conjuntos de datos empleados

Conjunto	Patrones	Entradas	Clases	Uso
XOR de 2 entradas	4	2	2	numeral 3
XOR de 3 entradas	8	3	2	numeral 3
Iris	150	4	3	numeral 4
Wine	178	13	3	numeral 5
Breast Cancer Wisconsin	569	30	2	numerales 5 y 6
Billetes	1372	4	2	numeral 5

Las particiones mezclan los índices con semilla fija y cortan en 60-40, 70-30, 80-20 y 90-10, como en el Taller 1. Las entradas se escalan al rango cero a uno con los mínimos y máximos del entrenamiento. Cuando hace falta validación, para la curva de error o para la parada temprana, se aparta una quinta parte del entrenamiento con una segunda semilla, y la exactitud se mide siempre sobre un conjunto de prueba que la red no vio. El Perceptrón y el Adaline no usan validación y entrenan con el conjunto de entrenamiento completo.

4. Resultados

4.1 Problema XOR

La Tabla 3 compara los tres modelos. El Perceptrón y el Adaline no convergen en ninguna semilla y quedan en promedio en 30 por ciento con dos entradas, frente al 25 por ciento del Taller 1 con

una sola semilla, y cerca del 50 con tres, el nivel de responder siempre la misma clase. El MLP con cuatro u ocho neuronas ocultas resuelve la XOR de dos entradas en todas las semillas, con 4117 y 3443 épocas en promedio para bajar el error por debajo de 0.001 con tasa 0.3 y sin momento. Con dos neuronas, el mínimo teórico, las cinco semillas clasifican bien pero una no alcanza el error objetivo. Con tres entradas solo la capa de ocho neuronas converge en la mayoría de las semillas.

Tabla 3: XOR de dos y tres entradas. Promedio de cinco semillas. El Perceptrón y el Adaline usan los parámetros del Taller 1, el MLP la tasa 0.3 sin momento. Convergió es el porcentaje de semillas que alcanzan el error objetivo, y un promedio de 5000 épocas indica que ninguna lo alcanzó

Entradas	Modelo	Neuronas ocultas	Épocas	Convergió (%)	Exactitud (%)
2	Perceptrón	ninguna	100.0	0	30.0
	Adaline	ninguna	200.0	0	30.0
	MLP	2	4406.0	80	100.0
	MLP	4	4116.6	100	100.0
	MLP	8	3443.0	100	100.0
3	Perceptrón	ninguna	100.0	0	47.5
	Adaline	ninguna	200.0	0	50.0
	MLP	2	5000.0	0	62.5
	MLP	4	4187.8	40	75.0
	MLP	8	3122.8	80	92.5

La Tabla 4 aísla el momento y la activación. Con cuatro neuronas y momento 0.9 la XOR de dos entradas converge en 436 épocas en lugar de 4117, y la de tres pasa de dos semillas convergidas a cinco. La tangente hiperbólica también acelera, 880 épocas en dos entradas, y la ReLU resulta poco fiable, tres de cinco semillas en dos entradas y ninguna en tres. La red de dos capas ocultas con tanh y sigmoide es la más rápida de esta tabla, 149 y 260 épocas, aunque como combina segunda capa, tanh y momento la tabla no permite separar cuánto aporta cada uno. La Figura 1 muestra la curva de error del MLP con momento 0.9 frente a la del Adaline.

Tabla 4: Efecto del momento y de la activación oculta sobre la XOR, y una red de dos capas ocultas con activación distinta en cada una. Promedio de cinco semillas, Convergió como en la Tabla 3

Entradas	Ocultas	Activación oculta	Momento	Épocas	Convergió (%)	Exactitud (%)
2	4	sigmoide	0	4116.6	100	100.0
	4	sigmoide	0.5	2087.6	100	100.0
	4	sigmoide	0.9	435.8	100	100.0
	4	tanh	0	879.8	100	100.0
	4	ReLU	0	2416.4	60	90.0
	[4 4]	tanh, sigmoide	0.9	149.0	100	100.0
3	4	sigmoide	0	4187.8	40	75.0
	4	sigmoide	0.5	4281.2	20	95.0
	4	sigmoide	0.9	1231.4	100	100.0
	4	tanh	0	1516.0	80	90.0
	4	ReLU	0	5000.0	0	87.5
	[4 4]	tanh, sigmoide	0.9	259.6	100	100.0

4.2 Tasa de aprendizaje, momento y pesos iniciales sobre la XOR

La guía de laboratorio del curso (2007) propone variar la tasa entre 0.1 y 2 con momento cero y el momento entre 0 y 1 con tasa 0.5. La Figura 2 reproduce ese barrido con cuatro neuronas ocultas

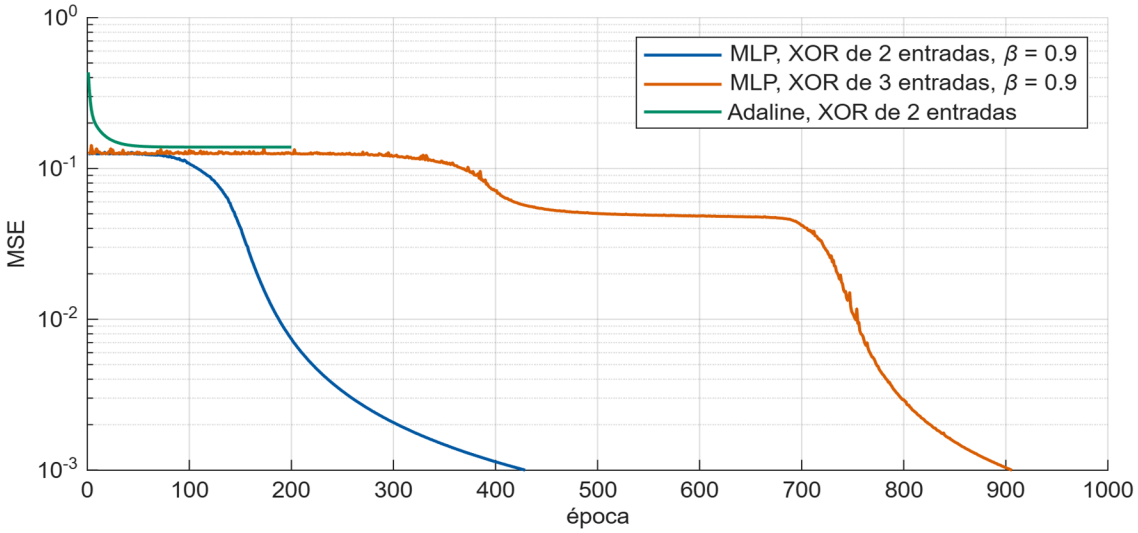


Figura 1: Curvas de error de entrenamiento sobre la XOR de dos y tres entradas con una de las cinco semillas, en escala logarítmica. El MLP con cuatro neuronas ocultas y momento 0.9 llega al error objetivo en ambos casos, y el Adaline, cuyo error es el acumulado durante cada época, se estanca cerca de 0.139, apenas por encima del piso de 0.125 de la mejor función lineal, la que responde 0.5 en los cuatro patrones

y cinco semillas, y agrega la escala de los pesos iniciales, que `init_mlp` fija y de la que depende que la red arranque. Con tasas 0.1 y 0.2 ninguna semilla llega al objetivo en 5000 épocas. De 0.3 en adelante todas convergen y cada vez más rápido, de 4117 épocas a 736 con tasa 2, sin que la tasa grande desestabilice el entrenamiento en estos cuatro patrones. El momento reduce las épocas de forma sostenida, de 2565 sin momento a 266 con 0.9, y con momento 1 el incremento anterior nunca se amortigua y ninguna semilla converge.

La escala inicial tiene un rango útil estrecho. Con 0.1 el error no baja del objetivo en 5000 épocas, probablemente porque las neuronas ocultas arrancan casi iguales. Entre 0.5 y 2 convergen todas las semillas. Con 5 solo convergen tres de cinco, probablemente porque las sumas ponderadas caen en la zona plana de la sigmoide, donde la derivada $f(1-f)$ es casi cero.

4.3 Activación en la capa oculta frente a la capa de salida

El numeral 1 pide elegir por separado la activación de las capas ocultas y la de salida, y la Tabla 5 muestra qué cambia cada elección. La capa de salida fija el factor f' del δ de la ecuación (2), igual a uno si es lineal, a lo sumo 0.25 si es sigmoide y a lo sumo uno si es tanh, y decide si la salida queda acotada. Las dos capas cambian el número de épocas. En la XOR la combinación más rápida fue tanh en la oculta con salida sigmoide, 95 épocas frente a 436 de sigmoide con sigmoide. Con salida tanh o lineal sobre una oculta tanh, y con salida lineal sobre una oculta ReLU, ninguna semilla llega al objetivo, mientras que las mismas salidas sobre una oculta sigmoide sí convergen. El fallo depende de la combinación y no de la salida sola. Con salida sigmoide la ReLU oculta vuelve a ser la menos fiable, 60 por ciento de semillas convergidas. En Breast Cancer las nueve combinaciones quedan entre 95.3 y 97.7 por ciento, una diferencia de cuatro tumores de 171 con una sola semilla, así que ahí la elección pesa mucho menos que en la XOR.

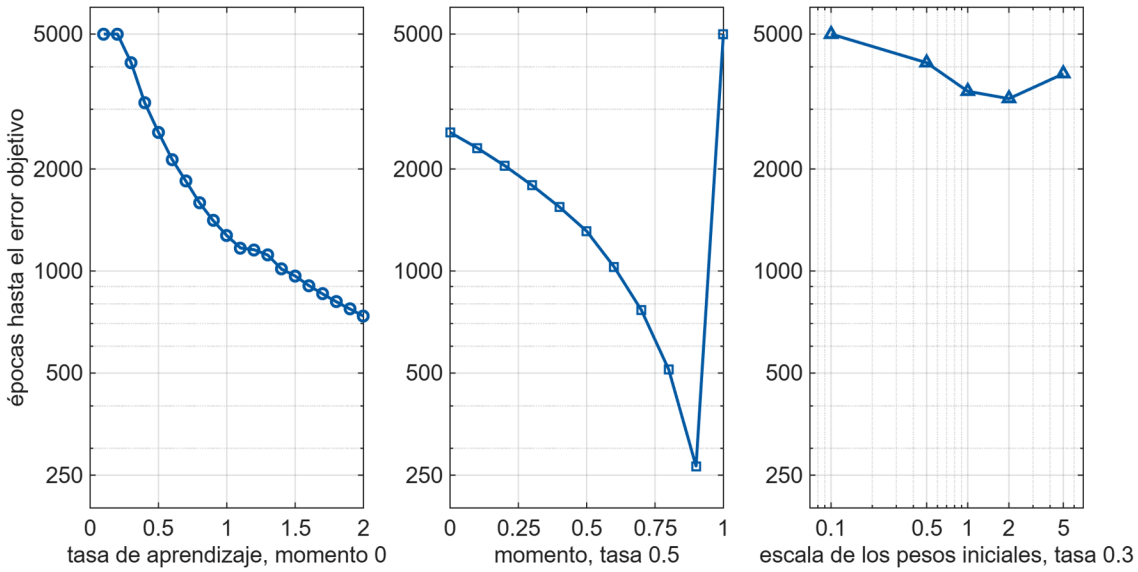


Figura 2: Épocas hasta el error objetivo sobre la XOR de dos entradas al variar la tasa de aprendizaje, el momento y la escala de los pesos iniciales, cuatro neuronas ocultas y promedio de cinco semillas. Un valor de 5000 indica que no se alcanzó el objetivo. Escala logarítmica en el eje vertical y en el horizontal del tercer panel, que usa tasa 0.3

Tabla 5: Activación de la capa oculta frente a la de salida. XOR de dos entradas con cuatro neuronas ocultas, momento 0.9 y cinco semillas, y Breast Cancer 70-30 con ocho neuronas ocultas, sin momento, parada tras 30 épocas sin mejora y una semilla. Convergió como en la Tabla 3

		XOR		Breast Cancer
Activación oculta	**Activación de salida**	**Épocas**	**Convergió (%)**	**Exactitud (%)**
sigmoide	sigmoide	435.8	100	97.7
	tanh	1011.2	100	97.1
	lineal	139.6	100	97.1
tanh	sigmoide	95.0	100	97.7
	tanh	5000.0	0	96.5
	lineal	5000.0	0	97.7
ReLU	sigmoide	2037.8	60	97.1
	tanh	3009.8	40	97.1
	lineal	5000.0	0	95.3

4.4 Clasificación multiclase en Iris

La Tabla 6 recoge las doce combinaciones sobre la partición 70-30, con 84 flores de entrenamiento, 21 de validación y 45 de prueba. Ninguna alcanza el error objetivo en 1000 épocas. El MSE de entrenamiento termina entre 0.005 y 0.017, porque *versicolor* y *virginica* se traslapan. Nueve configuraciones dan 97.8 por ciento, una sola flor mal clasificada, una *versicolor* tomada por *virginica*. Con tasa 0.05 las tres redes de una capa bajan a 95.6, porque a las 1000 épocas no terminan de ajustar esa frontera.

La mejor configuración se elige por el error de validación, sin mirar la prueba, y resulta la de doce neuronas con tasa 0.05, cuya matriz de confusión está en la Tabla 7. El menor error de validación no dio la mayor exactitud, y sobre 45 flores la diferencia entre configuraciones es una sola flor. La

Tabla 11 del Anexo A reúne las doce matrices.

Tabla 6: Iris sobre la partición 70-30. 1000 épocas en todas las configuraciones, exactitud sobre las 45 flores de prueba. Los segundos son los de una ejecución y cambian de una a otra

Neuronas ocultas	η	Segundos	MSE entrenamiento	MSE validación	Exactitud (%)
3	0.05	10.131	0.0159	0.0302	95.6
3	0.1	9.326	0.0114	0.0375	97.8
3	0.3	8.969	0.0106	0.0620	97.8
6	0.05	7.890	0.0166	0.0305	95.6
6	0.1	8.562	0.0115	0.0348	97.8
6	0.3	8.994	0.0066	0.0444	97.8
12	0.05	9.839	0.0167	0.0287	95.6
12	0.1	9.321	0.0117	0.0347	97.8
12	0.3	9.495	0.0095	0.0591	97.8
[8 4]	0.05	12.867	0.0109	0.0343	97.8
[8 4]	0.1	12.085	0.0068	0.0436	97.8
[8 4]	0.3	12.122	0.0053	0.0444	97.8

Tabla 7: Matriz de confusión sobre Iris de la configuración con menor error de validación, doce neuronas ocultas y tasa 0.05, 45 flores de prueba

Clase real	Predicha setosa	Predicha versicolor	Predicha virginica
setosa	14	0	0
versicolor	0	11	2
virginica	0	0	18

En la Figura 3 la tasa gobierna la forma de la curva más que el número de neuronas. Con 0.05 el error todavía baja a las 1000 épocas, con 0.1 se estabiliza cerca de la época 300, y con 0.3 llega antes pero oscila, y el error de validación se despega del de entrenamiento desde cerca de la época 100, un sobreajuste leve que no cuesta flores de prueba. La red de dos capas logra el menor error de entrenamiento y con 0.05 es la única que llega a 97.8, pero con 0.1 y 0.3 no mejora la exactitud.

4.5 Wine, Breast Cancer Wisconsin y billetes en distintas particiones

La Tabla 8 y la Figura 4 comparan los tres modelos sobre los tres conjuntos y las cuatro proporciones. Billetes se agregó por ser el conjunto del Taller 1. Sus cifras de Perceptrón y Adaline quedan cerca de las de allí sin coincidir, porque todos los modelos se escalan con las cotas del entrenamiento sin la parte de validación, y el orden de los patrones cambia.

En Wine el Perceptrón converge en todas las particiones entre 25 y 52 épocas y aun así el MLP lo supera en tres de cuatro. El Perceptrón se detiene en el primer hiperplano que separa el entrenamiento, por cerca que quede de los patrones, mientras el MLP sigue reduciendo el error hasta que la validación deja de mejorar. La excepción es 90-10, donde el Perceptrón acierta los 18 vinos y el MLP clasifica mal uno.

En Breast Cancer el Perceptrón nunca converge en 100 épocas, señal de clases traslapadas. El MLP lo supera en las tres primeras particiones y lo iguala en 90-10, y frente al Adaline solo pierde en 80-20 por un tumor de 114. En billetes la ventaja del MLP es consistente, entre 98.5 y 100 por ciento frente a un máximo de 99.0 del Perceptrón y 96.5 del Adaline. El Adaline queda en medio en Wine y último en billetes, porque el hiperplano que minimiza el error cuadrático no es el que mejor separa las clases.

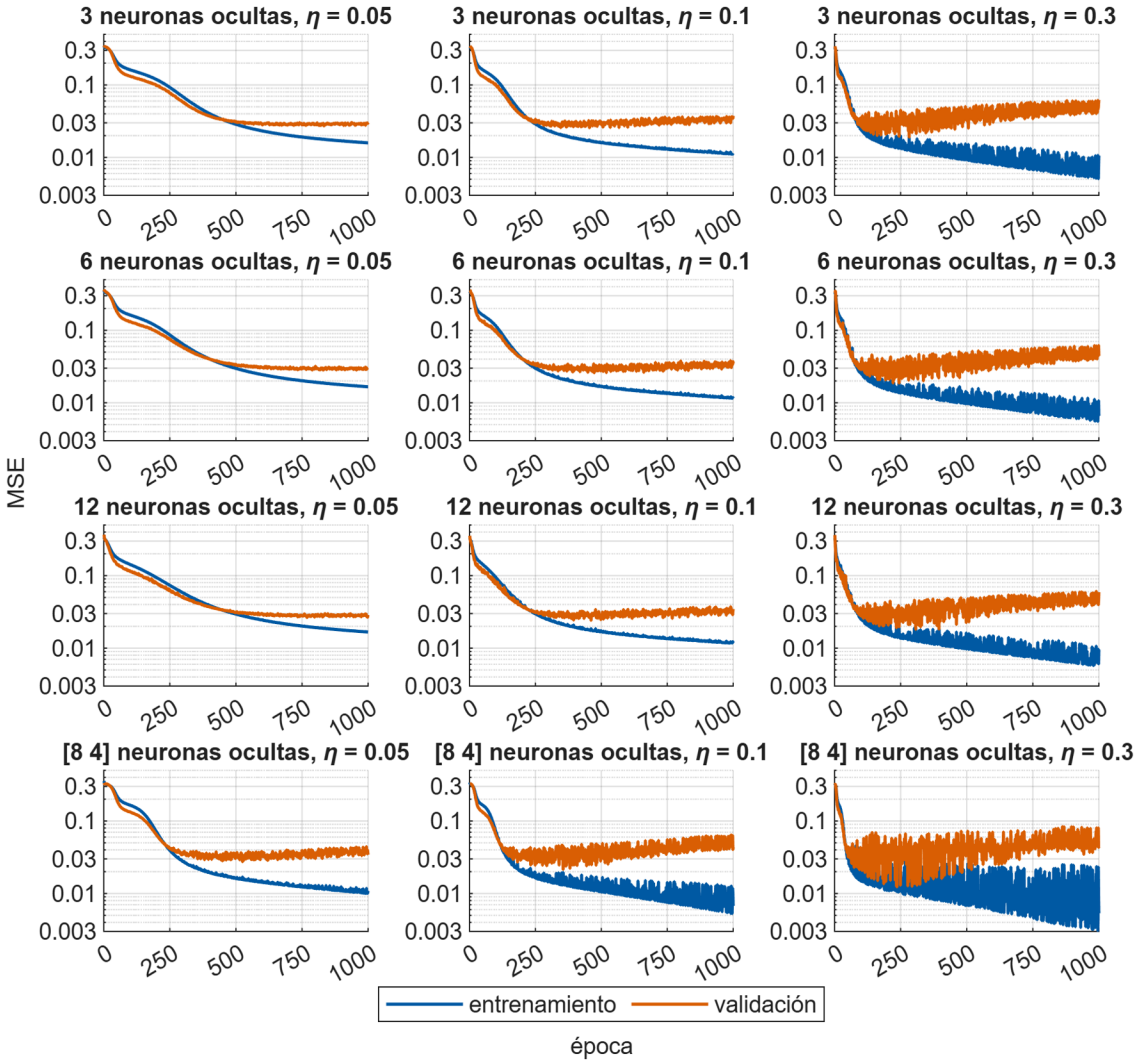


Figura 3: Curvas de error de entrenamiento y de validación sobre Iris para las cuatro arquitecturas y las tres tasas de aprendizaje, en escala logarítmica

El costo es el tiempo. Cada entrenamiento del MLP tarda segundos y los del Perceptrón y el Adaline centésimas o décimas de segundo, entre uno y dos órdenes de magnitud menos, porque en el MLP cada patrón atraviesa dos capas en cada sentido y la red evalúa la validación en cada época. En Wine y Breast Cancer la parada temprana detiene al MLP entre las épocas 82 y 180, 30 después de su mejor punto de validación. En billetes el MLP no llega a usarla, porque baja del error objetivo entre las épocas 105 y 146.

4.6 Análisis de sobreajuste (overfitting)

La Figura 5 muestra ambos errores sobre Breast Cancer durante 1500 épocas para cuatro números de neuronas ocultas, sin parada temprana, y la Tabla 9 resume las cifras. Los cuatro paneles tienen la misma forma. El error de validación toca un mínimo entre 0.0068 y 0.0079 entre las épocas 85 y 165 y después sube hasta terminar entre 0.0187 y 0.0220, entre 2.4 y 3 veces el mínimo, mientras el de entrenamiento baja hasta quedar entre 0.0036 y 0.0038. Esa separación creciente es el sobreajuste.

La Tabla 10 aísla el efecto del número de épocas con ocho neuronas ocultas. Cada entrenamiento

Tabla 8: Exactitud de prueba según proporción de partición, con las épocas ejecutadas y el tiempo de entrenamiento en segundos. El MLP usa ocho neuronas ocultas y se detiene por 30 épocas sin mejora en validación o por error objetivo. Las 200 épocas del Adaline en los tres conjuntos y las 100 del Perceptrón en Breast Cancer y billetes son el tope, sin convergencia. Los tiempos son los de una ejecución y cambian de una a otra

		MLP			Perceptrón			Adaline		
Conjunto	**Partición**	**%**	**Épocas**	**s**	**%**	**Épocas**	**s**	**%**	**Épocas**	**s**
Wine	60-40	98.6	149	1.466	95.8	52	0.017	97.2	200	0.051
	70-30	98.1	180	1.988	92.5	32	0.028	96.2	200	0.076
	80-20	100.0	148	1.913	94.4	25	0.011	97.2	200	0.066
	90-10	94.4	151	2.202	100.0	35	0.010	94.4	200	0.061
Breast Cancer	60-40	97.4	82	2.419	96.1	100	0.028	95.2	200	0.061
	70-30	97.7	94	2.891	95.3	100	0.034	97.1	200	0.055
	80-20	95.6	102	3.563	93.0	100	0.029	96.5	200	0.092
	90-10	98.2	118	4.175	98.2	100	0.043	98.2	200	0.062
Billetes	60-40	99.5	132	8.035	96.9	100	0.055	96.5	200	0.098
	70-30	100.0	146	11.490	99.0	100	0.052	95.9	200	0.109
	80-20	99.3	125	9.993	96.7	100	0.077	96.4	200	0.112
	90-10	98.5	105	9.531	97.8	100	0.128	95.6	200	0.188

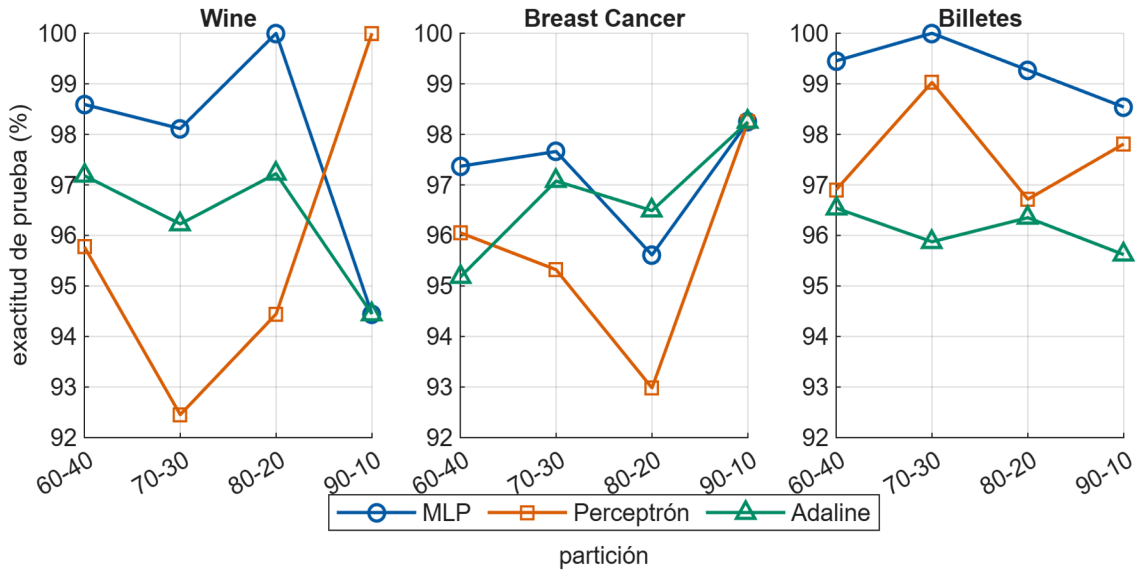


Figura 4: Exactitud de prueba de los tres modelos según la proporción de partición en los tres conjuntos reales, con el mismo rango vertical en los tres paneles. En 90-10 los marcadores coinciden donde los modelos empatan

más largo baja el error de entrenamiento y sube el de validación, y la exactitud de prueba cae de 95.9 por ciento a las 200 épocas a 94.7 a las 1000 y 1500, es decir dos tumores más mal clasificados por seguir entrenando.

La parada temprana guarda los pesos del mejor punto y detiene el entrenamiento tras 50 épocas sin mejorar el mínimo de validación. Termina entre las épocas 135 y 215 y la exactitud de prueba sube a 97.7 por ciento en los cuatro casos, cuatro tumores fallados de 171, frente a 94.2 a 94.7 sin parada. El número de neuronas apenas cambia el resultado. Con 2 y con 64 neuronas el mínimo de validación vale lo mismo, y la red de 64 solo lo alcanza antes.

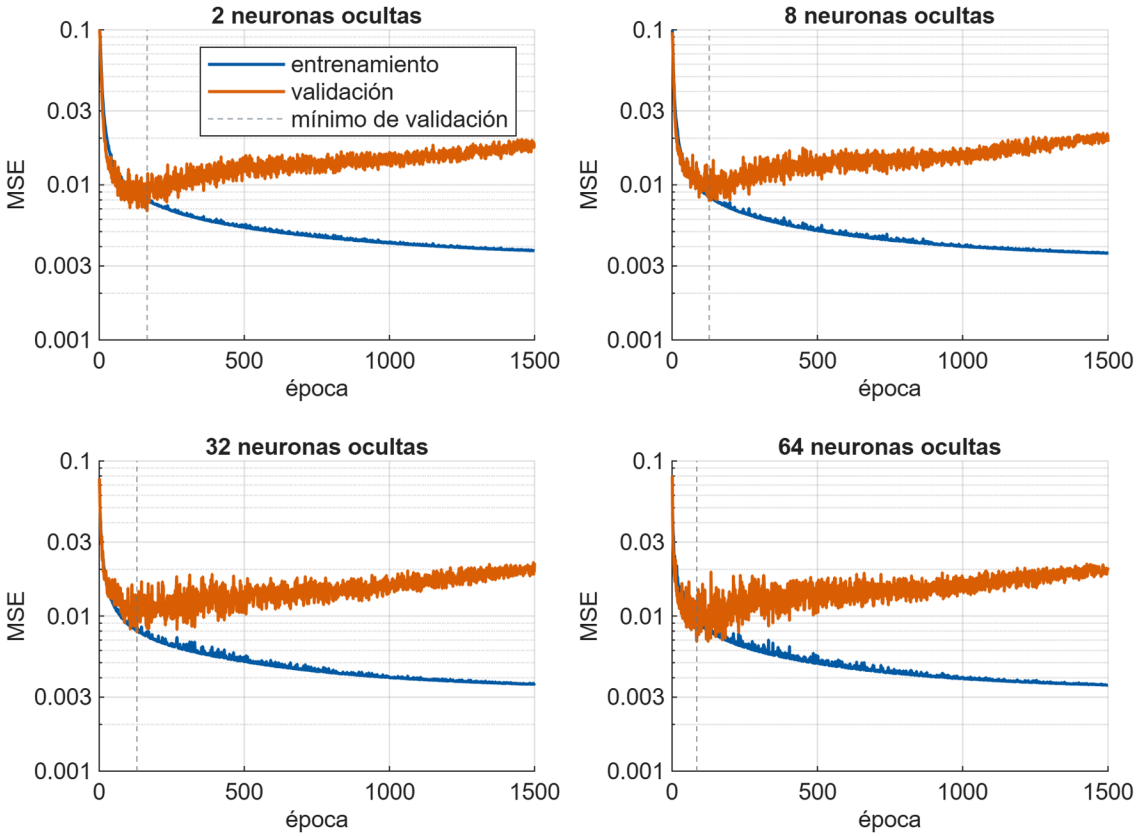


Figura 5: Error de entrenamiento y de validación sobre Breast Cancer para cuatro números de neuronas ocultas. La línea discontinua marca la época de mínimo error de validación. Escala logarítmica

Tabla 9: Sobreajuste en Breast Cancer, partición 70-30. Sin parada temprana el entrenamiento dura 1500 épocas, con parada temprana se detiene tras 50 épocas sin mejora en validación. Las abreviaturas ent., val. y mín. corresponden a entrenamiento, validación y mínimo

	Sin parada temprana, 1500 épocas					Con parada temprana	
Ocultas	**MSE ent.**	**MSE val.**	**Época mín.**	**MSE val. mín.**	**Exact. (%)**	**Épocas**	**Exact. (%)**
2	0.0038	0.0187	165	0.0068	94.7	215	97.7
8	0.0036	0.0189	128	0.0079	94.7	178	97.7
32	0.0037	0.0220	130	0.0079	94.2	180	97.7
64	0.0036	0.0206	85	0.0068	94.7	135	97.7

Tabla 10: Efecto del número de épocas con ocho neuronas ocultas sobre Breast Cancer, sin parada temprana

Épocas	**MSE entrenamiento**	**MSE validación**	**Exactitud de prueba (%)**
200	0.0071	0.0115	95.9
500	0.0051	0.0126	95.3
1000	0.0040	0.0146	94.7
1500	0.0036	0.0189	94.7

5. Análisis y discusión

5.1 Por qué el MLP resuelve la XOR y los modelos del Taller 1 no

El Perceptrón y el Adaline quedan en 30 y cerca de 50 por ciento porque una neurona traza un hiperplano y ninguno separa la XOR. El MLP transforma la entrada antes de decidir. Cada neurona oculta traza su hiperplano y la salida combina las regiones, así que el problema que llega a la salida ya es separable. La fila de dos neuronas confirma que dos rectas bastan, pues clasifica bien en las cinco semillas aunque una no baje el error por debajo de 0.001. Con cuatro u ocho neuronas la red dispone de más rectas de las necesarias y todas las semillas llegan al objetivo.

Con tres entradas hay ocho vértices de un cubo alternando clase y hacen falta más regiones. El momento fue el cambio de mayor efecto, pues con cuatro neuronas pasó de dos semillas convergidas a cinco. Con β 0.9 la corrección acumula impulso en la dirección que se repite época tras época, lo que ayuda a salir de mesetas como las que muestra la curva de tres entradas de la Figura 1.

5.2 Efecto del número de neuronas ocultas y de la tasa de aprendizaje

El valor útil de la tasa depende del problema. En la XOR sube hasta 2 sin perder estabilidad y reduce las épocas de 4117 a 736. En Iris, con clases traslapadas y datos no vistos, la tasa 0.05 no se ha estabilizado a las 1000 épocas y la tasa 0.3 ya hace oscilar el error y separa la validación del entrenamiento.

El número de neuronas ocultas importó menos de lo esperado. En Iris las cuatro arquitecturas dan la misma exactitud con 0.1 y 0.3, y en Breast Cancer los cuatro tamaños caen en un mínimo de validación muy parecido, entre 0.0068 y 0.0079. Solo en la XOR, donde la geometría exige un número mínimo de hiperplanos, el tamaño decide entre converger o no. En los conjuntos reales las neuronas adicionales no mejoran la clasificación, solo agregan pesos, y en la Tabla 6 el tiempo no sigue al número de neuronas entre tres y doce y solo sube con la segunda capa oculta.

5.3 Proporción de partición y generalización

Con una semilla por partición, ninguna proporción resultó mejor de forma consistente. En Wine el Perceptrón alcanza su máximo en 90-10 y el MLP en 80-20, en Breast Cancer el MLP se mueve entre 95.6 y 98.2 sin orden, y en billetes las cuatro dan entre 98.5 y 100. Lo que sí se repite es la fragilidad de las pruebas pequeñas. Con 18 vinos o 57 tumores un solo error mueve la exactitud entre 1.8 y 5.6 puntos, y por eso el 100 por ciento del Perceptrón en Wine 90-10 o del MLP en Wine 80-20 dice menos de lo que parece.

5.4 Evidencia de sobreajuste y su mitigación

La evidencia está en la Figura 5 y en la Tabla 10. A las 1500 épocas la red de ocho neuronas clasifica 1.2 puntos peor que a las 200, y las cuatro redes quedan entre 3.0 y 3.5 puntos por debajo de su versión detenida a tiempo, mientras su error de entrenamiento es unas cinco veces menor que el de validación.

La parada temprana lo corrigió en los cuatro casos y los llevó al mismo 97.7 por ciento, sin decidir de antemano el número de épocas, porque lo decide la validación. El precio es apartar 80 de los 398 patrones de entrenamiento, así que la red aprende con 318. Subir de 2 a 64 neuronas no agravó el sobreajuste, solo lo adelantó de la época 165 a la 85. La causa aquí es el exceso de épocas y el remedio es parar a tiempo, no reducir la red.

6. Conclusiones

El perceptrón multicapa resolvió la XOR de dos entradas al 100 por ciento en todas las semillas con cuatro u ocho neuronas ocultas, y la de tres entradas al agregar momento 0.9. El Perceptrón y el Adaline, con los parámetros del Taller 1, quedaron en promedio en 30 por ciento con dos entradas y entre 47.5 y 50 con tres. Dos neuronas ocultas bastan para clasificar, aunque con ellas una de cinco semillas no llega al error objetivo.

El momento fue el parámetro de mayor efecto sobre la velocidad. Con 0.9 la XOR de dos entradas pasó de 4117 a 436 épocas y la de tres de dos semillas convergidas a cinco. El barrido de la guía mostró que la tasa llega a 2 sin desestabilizar, que el momento 1 impide converger y que la escala inicial solo lleva todas las semillas al objetivo entre 0.5 y 2. Entre las activaciones, la salida sigmoide fue la única que alcanzó el objetivo con las tres activaciones ocultas, aunque con ReLU solo en tres de cinco semillas, y la salida tanh o lineal sobre una oculta tanh no lo alcanzó en ninguna semilla con momento 0.9.

Sobre Iris las doce configuraciones dieron 95.6 o 97.8 por ciento, y la tasa determinó la forma de la curva más que el número de neuronas o de capas. Por error de validación la mejor fue doce neuronas con tasa 0.05, y la tasa 0.1 dio curvas sin oscilación que se estabilizan cerca de la época 300. Sobre 45 flores ninguna se distingue por exactitud.

En los conjuntos reales el MLP superó al Perceptrón en diez de las doce particiones y al Adaline en nueve, y solo perdió dos veces, en Wine 90-10 frente al Perceptrón y en Breast Cancer 80-20 frente al Adaline, cada vez por un solo patrón de prueba. La ventaja más clara estuvo en billetes, entre 98.5 y 100 por ciento. El precio fue un tiempo de entrenamiento entre uno y dos órdenes de magnitud mayor.

El sobreajuste apareció en Breast Cancer como una subida sostenida del error de validación a partir de un mínimo situado entre las épocas 85 y 165, mientras el de entrenamiento seguía bajando. Costó 1.2 puntos de exactitud entre 200 y 1500 épocas, y su magnitud no dependió del número de neuronas ocultas entre 2 y 64, aunque la red más grande llegó antes al mínimo. La parada temprana por 50 épocas sin mejora lo corrigió y elevó la exactitud de prueba de entre 94.2 y 94.7 a 97.7 por ciento en los cuatro casos.

Ninguna proporción de partición resultó mejor de forma consistente. Las particiones 90-10 evalúan sobre muy pocos patrones, y el 100 por ciento del Perceptrón en Wine 90-10 equivale a un vino más acertado que el MLP.

7. Referencias

- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- Haykin, S. (2009). *Neural Networks and Learning Machines* (3.ª ed.). Pearson.
- Redes neuronales, algoritmo Backpropagation (2007). Guía de laboratorio, material del curso Inteligencia Computacional Aplicada, Universidad Distrital Francisco José de Caldas.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
- Aeberhard, S., & Forina, M. (1992). Wine. UCI Machine Learning Repository.
- Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1993). Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository.

- Lohweg, V. (2012). Banknote Authentication. UCI Machine Learning Repository.

A. Tablas complementarias

Tabla 11: Matrices de confusión de las doce configuraciones sobre Iris. Cada celda es el número de flores de la clase real en fila asignadas a la clase en columna, con las clases en el orden setosa, versicolor, virginica

Ocultas	η	se-se	se-ve	se-vi	ve-se	ve-ve	ve-vi	vi-se	vi-ve	vi-vi
3	0.05	14	0	0	0	11	2	0	0	18
3	0.1	14	0	0	0	12	1	0	0	18
3	0.3	14	0	0	0	12	1	0	0	18
6	0.05	14	0	0	0	11	2	0	0	18
6	0.1	14	0	0	0	12	1	0	0	18
6	0.3	14	0	0	0	12	1	0	0	18
12	0.05	14	0	0	0	11	2	0	0	18
12	0.1	14	0	0	0	12	1	0	0	18
12	0.3	14	0	0	0	12	1	0	0	18
[8 4]	0.05	14	0	0	0	12	1	0	0	18
[8 4]	0.1	14	0	0	0	12	1	0	0	18
[8 4]	0.3	14	0	0	0	12	1	0	0	18

B. Código de los bloques completados

Se reproducen completos los cuatro bloques que la plantilla dejaba pendientes y la función de entrenamiento con parada temprana que se agregó para los experimentos. Ningún nombre ni argumento de las funciones de la plantilla fue modificado.

Código 1: Inicialización de pesos y sesgos

```
function net = init_mlp(cfg)
    % Pesos aleatorios pequeños para que las neuronas de una capa no arranquen
    % iguales y la sigmoide no sature. Sesgos en cero. Los incrementos anteriores
    % en cero porque el momento aún no tiene nada que sumar.
    net = struct();
    net.W = {};
    net.b = {};
    net.dW_prev = {};
    net.db_prev = {};
    layer_sizes = [cfg.n_inputs, cfg.hidden_layers, cfg.n_outputs];
    escala = 0.5;  if isfield(cfg, 'init_scale'), escala = cfg.init_scale; end   % 0.5 es el valor de la
     plantilla
    for k = 1:numel(layer_sizes) - 1
        net.W{k} = randn(layer_sizes(k+1), layer_sizes(k)) * escala;
        net.b{k} = zeros(layer_sizes(k+1), 1);
        net.dW_prev{k} = zeros(size(net.W{k}));
        net.db_prev{k} = zeros(size(net.b{k}));
    end
end
```

Código 2: Propagación hacia adelante

```
function [y, cache] = forward_mlp(net, x, cfg)
    % net_k = W_k a_(k-1) + b_k y a_k = f(net_k), capa por capa. Se guardan net y a
    % de cada capa porque backward_mlp los necesita.
    a{1} = x;
    L = numel(net.W);
    for k = 1:L
        netv{k} = net.W{k} * a{k} + net.b{k};
        if k == L
            a{k+1} = activation(netv{k}, cfg.act_output);
        else
            a{k+1} = activation(netv{k}, hidden_act(cfg, k));
        end
    end
    y = a{end};
    cache.a = a;
    cache.netv = netv;
end

function tipo = hidden_act(cfg, k)
    % Activación de la capa oculta k, una para todas o una por capa si act_hidden es celda.
    if iscell(cfg.act_hidden), tipo = cfg.act_hidden{k}; else, tipo = cfg.act_hidden; end
end
```

Código 3: Retropropagación del error y actualización con momento

```
function net = backward_mlp(net, cache, x, d, cfg)
    % delta de salida (d - y) f'(net), delta oculta (W' delta_siguiente) f'(net),
    % y después dW = eta delta a' + beta dW_anterior, en pesos y sesgos.
    L = numel(net.W);
    delta = cell(1, L);
    delta{L} = (d - cache.a{L+1}) .* activation_deriv(cache.netv{L}, cfg.act_output);
    for k = L-1:-1:1
        delta{k} = (net.W{k+1}' * delta{k+1}) .* activation_deriv(cache.netv{k}, hidden_act(cfg, k));
    end
    for k = 1:L
        dW = cfg.eta * delta{k} * cache.a{k}' + cfg.momentum * net.dW_prev{k};
        db = cfg.eta * delta{k} + cfg.momentum * net.db_prev{k};
        net.W{k} = net.W{k} + dW;
        net.b{k} = net.b{k} + db;
        net.dW_prev{k} = dW;
        net.db_prev{k} = db;
    end
end
```

Código 4: Derivadas de las funciones de activación

```
function dy = activation_deriv(net_in, tipo)
    % Derivadas evaluadas en net_in. La ReLU toma derivada cero en net = 0.
    switch tipo
        case 'sigmoid'
            f = 1 ./ (1 + exp(-net_in));
            dy = f .* (1 - f);
        case 'tanh'
            dy = 1 - tanh(net_in).^2;
        case 'relu'
            dy = double(net_in > 0);
        case 'linear'
            dy = ones(size(net_in));
        otherwise
            error('Función de activación no reconocida: %s', tipo);
    end
end
```

Código 5: Entrenamiento con error de validación y parada por épocas sin mejora

```
function [net, hist] = train_mlp(Xtr, Dtr, Xva, Dva, cfg)
    % Ciclo de la sección 8 con el error medido al final de cada época sobre todo
    % el conjunto, error de validación por época y parada
    % temprana cuando la validación lleva cfg.patience épocas sin mejorar.
    % Sin conjunto de validación la paciencia vigila el error de entrenamiento.
    if iscell(cfg.act_hidden)
        assert(numel(cfg.act_hidden) == numel(cfg.hidden_layers), 'act_hidden necesita una activación por capa'
    );
    end
    rng(cfg.seed);
    net = init_mlp(cfg);
    hist.train = zeros(cfg.max_epochs, 1);  hist.val = hist.train;
    mejor = Inf;  sin_mejora = 0;  mejor_net = net;  mejor_epoca = 0;
    for epoch = 1:cfg.max_epochs
        for p = randperm(size(Xtr, 1))
            [~, cache] = forward_mlp(net, Xtr(p, :)', cfg);
            net = backward_mlp(net, cache, Xtr(p, :)', Dtr(p, :)', cfg);
        end
        hist.train(epoch) = mse_mlp(net, Xtr, Dtr, cfg);
        if ~isempty(Xva), hist.val(epoch) = mse_mlp(net, Xva, Dva, cfg); end
        if hist.train(epoch) <= cfg.target_error, break; end
        if cfg.patience > 0
            if isempty(Xva), vigilado = hist.train(epoch); else, vigilado = hist.val(epoch); end
            if vigilado < mejor
                mejor = vigilado;  sin_mejora = 0;  mejor_net = net;  mejor_epoca = epoch;
            else
                sin_mejora = sin_mejora + 1;
                if sin_mejora >= cfg.patience, break; end
            end
        end
    end
    hist.converged = hist.train(epoch) <= cfg.target_error;
    if cfg.patience > 0 && ~hist.converged
        net = mejor_net;
    else
        mejor_epoca = epoch;
    end
    hist.train = hist.train(1:epoch);  hist.val = hist.val(1:epoch);
    hist.epochs = epoch;  hist.best_epoch = mejor_epoca;
end
```